

Логические операции в NASM



Повторение. Флаги

- Какие флаги бывают, за что отвечает каждый?



Повторение. Флаги

- Какие флаги бывают, за что отвечает каждый?
 - **ZF == 1** – результат последней операции равен 0



Повторение. Флаги

- Какие флаги бывают, за что отвечает каждый?
 - **ZF == 1** – результат последней операции равен 0
 - **SF == 1** – результат последней операции меньше 0



Повторение. Флаги

- Какие флаги бывают, за что отвечает каждый?
 - **ZF == 1** – результат последней операции равен 0
 - **SF == 1** – результат последней операции меньше 0
 - **CF == 1** – результат последней операции не влез в unsigned число: заняли 1 или запомнили 1 за пределами числа



Повторение. Флаги



- Какие флаги бывают, за что отвечает каждый?
 - **ZF == 1** – результат последней операции равен 0
 - **SF == 1** – результат последней операции меньше 0
 - **CF == 1** – результат последней операции не влез в unsigned число: заняли 1 или запомнили 1 за пределами числа
 - **OF == 1** – результат последней операции не влез в signed число: при сложении положительных чисел получили отрицательный результат или при вычитании из отрицательного числа положительного получили положительный результат

Повторение. Умножение

MUL <REG / MEM> – умножает EAX на свой операнд и записывает результат в EDX:EAX

То есть имеем:

- один операнд явный
- второй – всегда регистр EAX / AX / AL

Результат ложится в два регистра разом:

- Нижняя половина в EAX / AX
- Верхняя половина в EDX / DX

MUL – беззнаковое умножение

IMUL – знаковое умножение



Повторение. Деление

DIV <REG / MEM> – делит EDX:EAX на свой операнд. В EAX кладет частное, в EDX – остаток

То есть имеем:

- делимое – EDX:EAX
- делитель – операнд

Результат ложится в два регистра разом:

- частное в EAX
- остаток в EDX

DIV – беззнаковое деление

IDIV – знаковое



Битовые операции

AND <OP1>, <OP2>

OR <OP1>, <OP2>

XOR <OP1>, <OP2>

NOT <OP>



```
section .text
main:
    put_values

    ; AND
    and eax, ebx           ; 1100 & 1010 = 1000 (8)
    print_hex_newline

    ; OR
    put_values
    or  eax, ebx           ; 1100 | 1010 = 1110 (14 = e)
    print_hex_newline

    ; XOR
    put_values
    xor eax, ebx           ; 1100 ^ 1010 = 0110 (6)
    print_hex_newline

    ; NOT
    put_values
    not eax                ; ~1100 = ffffffff3
    print_hex_newline

    ret
```

Битовые сдвиги. SHL, SHR

Классический битовый сдвиг, который мы знаем из C

`SHL eax, 1 == eax << 1`



```
; -----  
; SHL - shift left LOGICAL  
; -----  
mov eax, 0x00000011      ; 0001 0001b  
shl eax, 1                ; 0010 0010b == 0x22  
call io_print_hex  
call io_newline  
  
; -----  
; SHR - shift right LOGICAL  
; -----  
mov eax, 0x00000011      ; 0001 0001b  
shr eax, 1                ; 0000 1000b = 0x8  
call io_print_hex  
call io_newline
```

Битовые сдвиги. SAL, SAR

- Сдвиги с учетом знака
- SAL – это то же самое, что SHL, знаковый бит всегда сдвигается в CF
- SAR – сдвиг вправо с учетом знака. Появившееся место заполняется знаковым битом исходного числа



Битовые сдвиги. SAL, SAR

```
; -----  
; SAR - shift right ARITHMETIC  
; -----  
mov eax, 0x80000011      ; negative value = -2 == 1000 0000 0000 0000 0000 0000 0001 0001  
sar eax, 1               ; highest bit ( = sign bit) is saved == 0xc0000008  
call io_print_hex  
call io_newline  
  
; -----  
; SAL - shift left ARITHMETIC  
; -----  
mov eax, 0x00000011  
sal eax, 1  
call io_print_hex  
call io_newline
```



Битовые сдвиги. ROR, ROL

- Циклические сдвиги
- Нет прямой аналогии в языке C
- **ROR 0b01101, 1 -> 0b10110**
- **ROL 0b01101, 1 -> 0b11010**

Битовые сдвиги. ROR, ROL

- Циклические сдвиги
- Нет прямой аналогии в языке C
- **ROR 0b01101, 1 -> 0b10110**
- **ROL 0b01101, 1 -> 0b11010**





БИТОВЫЕ СДВИГИ. ROR, ROL

```
; -----  
; ROL - Rotate Left = cyclic shift left  
; -----  
mov eax, 0x80000001      ;  
rol eax, 1                ; highest bit becomes lowest one  
call io_print_hex  
call io_newline  
  
; -----  
; ROR - Rotate Right = cyclic shift right  
; -----  
mov eax, 0x80000001  
ror eax, 1  
call io_print_hex  
call io_newline
```

Битовые сдвиги. ROR, ROL

- запрогаем ROR на C. Компилятор сгенерирует нам три инструкции:
 1. **SHR value, shift** ; Сохраним в EAX за кадром
 2. **SHL value, 32 - shift** ; Сохраним в EBX за кадром
 3. **OR EAX, EBX**
- Эффективный ли это код?



```
unsigned int ror32(unsigned int value, unsigned int shift) {  
    return (value >> shift) | (value << (32 - shift));  
}
```


Битовые сдвиги. ROR, ROL

- запрогаем ROR на C. Компилятор сгенерирует нам три инструкции:
 1. **SHR value, shift** ; Сохраним в EAX за кадром
 2. **SHL value, 32 - shift** ; Сохраним в EBX за кадром
 3. **OR EAX, EBX**
- Эффективный ли это код? – Конечно нет!



```
unsigned int ror32(unsigned int value, unsigned int shift) {  
    return (value >> shift) | (value << (32 - shift));  
}
```

Битовые сдвиги. ROR, ROL

- запрогаем ROR на C. Компилятор сгенерирует нам три инструкции:
 1. **SHR value, shift** ; Сохраним в EAX за кадром
 2. **SHL value, 32 - shift** ; Сохраним в EBX за кадром
 3. **OR EAX, EBX**
- Эффективный ли это код? – Конечно нет!
- Можем ли мы что-то сделать с этим?



```
unsigned int ror32(unsigned int value, unsigned int shift) {  
    return (value >> shift) | (value << (32 - shift));  
}
```

Битовые сдвиги. ROR, ROL

- запрогаем ROR на C. Компилятор сгенерирует нам три инструкции:
 1. **SHR value, shift** ; Сохраним в EAX за кадром
 2. **SHL value, 32 - shift** ; Сохраним в EBX за кадром
 3. **OR EAX, EBX**
- Эффективный ли это код? – Конечно нет!
- Можем ли мы что-то сделать с этим?
- Давайте узнаем: <https://godbolt.org/z/n6hWseMfc>



```
unsigned int ror32(unsigned int value, unsigned int shift) {  
    return (value >> shift) | (value << (32 - shift));  
}
```

Битовые сдвиги. RCL, RCR

- Циклический сдвиг с учетом флага CF.
- CF “приклеивается” к числу и сдвигается вместе с ним



RCL - циклический сдвиг влево

